

Karim N Trucking

Invoicing System — Technical Documentation

Architecture, code walkthrough, and data model

This document explains how the Karim N Trucking invoicing system works end to end: the two files that make up the whole system, how they talk to each other, the data model, the business rules encoded in the code, and the security model. It's written for whoever maintains or extends this system next — including a future version of the person who built it.

1. Overview

The invoicing system lets owner-operator drivers submit biweekly invoices for the loads they've completed, and lets an operations team review, manage, and export that data. It has two portals inside one app:

- **Driver Portal** — a driver selects their name, enters a PIN, logs their loads for a pay period, previews the calculated totals, and submits.
- **Operations Portal** — PIN-protected. Lets staff view/filter all invoices, manage the driver roster (PINs, commission rates, company names), change system-wide settings, and export a master Excel workbook.

There is no framework and no build step. The frontend is a single static HTML file; the backend is a single Google Apps Script file deployed as a Web App. There is no traditional database — a Google Sheet *is* the database.

2. Architecture

```
Driver / Ops browser
  karim-invoicing-app.html (HTML + vanilla JS, no build step)
    |
    | fetch() - GET for reads, POST (text/plain body) for writes
    |           (text/plain avoids a CORS preflight request,
    |           since Apps Script doesn't handle OPTIONS)
    |
    v
Google Apps Script Web App
  karim-invoicing-backend.gs
    |
    +-- Google Sheet (Drivers / Invoice Summary / Invoice Loads tabs)
    +-- Google Drive (generated invoice PDFs, shared view-only links)
    +-- Gmail (GmailApp.sendEmail - notifies ops, cc's driver)
    +-- PropertiesService (ops PIN, tax-basis setting - key/value store)
```

The frontend holds no data of its own between page loads — every screen reflects what's in a single in-memory object (DATA), which is populated by one GET request when the app boots, and refreshed after every write.

3. Files In This Project

File	Purpose
karim-invoicing-app.html	The entire frontend — driver portal, ops portal, all screens, all logic. Only external dependency is SheetJS (xlsx)
karim-invoicing-backend.gs	The entire backend. Deployed as an Apps Script Web App. Acts as both the API and the database layer (via Google Sheets)
SETUP-INSTRUCTIONS.md	Deployment walkthrough — creating the Sheet, deploying the script, connecting the two, and hosting the HTML

4. Frontend Architecture

4.1 State

All data from the backend lives in one object:

```
let DATA = { drivers: [], invoices: [], settings: { opsPin: '1234', taxBasis: 'net' } };
```

UI state — which screen is showing, who's logged in, what stage a form is at — lives in a set of top-level let variables: VIEW, authedDriver, driverStage, driverTab, opsAuthed, opsTab, editingTicketNo, and a few others.

4.2 Rendering pattern

There's no framework, so rendering follows a simple, explicit pattern repeated for every screen:

- A screenX() function returns an HTML string for that screen, built from the current state.
- A matching wireX() function runs immediately after that HTML is inserted into the page, attaching event listeners (onclick, onchange, etc.) to the elements that now exist in the DOM.
- A single render() function is the only place that writes to #app's innerHTML — it looks at VIEW, calls the matching screen + wire pair, and that's the entire routing system.

```
function render() {
  const app = document.getElementById('app');
  if (VIEW === 'home') { app.innerHTML = screenHome(); wireHome(); }
  else if (VIEW === 'driverApp') { app.innerHTML = screenDriverApp(); wireDriverApp(); }
  else if (VIEW === 'opsApp') { app.innerHTML = screenOpsApp(); wireOpsApp(); }
  // ...
}
```

4.3 Avoiding lost focus while typing

Fully re-rendering a screen on every keystroke (as the pattern above would naively suggest) destroys and recreates the DOM node the user is typing in, which drops keyboard focus on every character. To avoid that, text inputs (pickup, price, reference number, drop-off text) use `onInput` handlers that mutate the underlying data array directly and only update a small, separate totals display — never the input's own DOM node. Structural changes (adding a load, adding a drop-off, changing a reference-number tab) do trigger a full rebuild of that section, which is fine because those aren't continuous-typing actions.

4.4 Shared calendar component

Every date field (period start, period end, each load's date) shares one calendar pop-over instance rather than each field having its own — the pop-over is created once, and `openDatePicker(anchorEl, currentValue, onSelect)` repositions it next to whichever button was clicked and hands back the selected date via a callback.

4.5 Networking

- `apiGetAll()` — a plain GET to the Apps Script URL, returns the full dataset as JSON.
- `apiPost(payload)` — POSTs JSON as a text/plain body (not `application/json`) specifically to avoid triggering a CORS preflight OPTIONS request, which Apps Script Web Apps don't handle.
- Both are wrapped with a 25-second timeout via `AbortController`, so a slow or unreachable backend fails with a clear message instead of hanging indefinitely.
- `loadAll()` de-duplicates concurrent calls — if it's already in flight, a second call reuses the same promise instead of firing a duplicate request.
- `performAction(payload, successMsg)` wraps every ops mutation (add/update/remove driver, settings changes) so failures always surface a toast message instead of failing silently.

5. Backend Architecture (Google Apps Script)

5.1 Entry points

Apps Script Web Apps expose exactly two entry points, and this project uses both:

```
function doGet(e) {  
  // Returns { success, drivers, invoices, settings } – the entire  
  // dataset the frontend needs, in one call.  
}  
  
function doPost(e) {  
  const payload = JSON.parse(e.postData.contents);  
  switch (payload.action) {  
    case 'submitInvoice': return submitInvoice(payload);  
    case 'updateInvoice': return updateInvoice(payload);  
    case 'addDriver':      return addDriver(payload);  
    case 'updateDriver':   return updateDriver(payload);  
    case 'removeDriver':   return removeDriver(payload);  
    case 'updateSettings': return updateSettings(payload);  
  }  
}
```

Every write action is a distinct branch under one POST endpoint, distinguished purely by an `action` field in the JSON body — there's no REST-style routing, just a switch statement.

5.2 Sheet-as-database pattern

`getOrCreateSheet(name, headers)` is the core primitive the whole backend is built on: it looks up a tab by name in the spreadsheet, and if it doesn't exist yet, creates it with a styled header row. Every read/write function calls this first, so the Sheet effectively self-initializes on first use — nothing needs to be manually set up beyond creating the blank spreadsheet itself.

5.3 Settings storage

The operations PIN and the tax-calculation-basis setting are *not* stored in the Sheet — they live in Apps Script's `PropertiesService`, a simple key/value store tied to the script project itself. This keeps them safe from accidental edits to the spreadsheet and avoids needing a dedicated settings tab for two values.

```
function readSettings() {  
  const props = PropertiesService.getScriptProperties();  
  return {  
    opsPin: props.getProperty('opsPin') || '1234',  
    taxBasis: props.getProperty('taxBasis') || 'net'  
  };  
}
```

5.4 PDF generation

Each invoice's PDF is generated without any external library or API — Apps Script can convert an HTML string directly to a PDF blob:

```
const blob = Utilities.newBlob(html, 'text/html', name + '.html')
    .getAs('application/pdf');
```

That blob is saved into a Drive folder (created on first use, same lazy pattern as the sheets), shared as "Anyone with the link — View" so it can be opened without a Google login, and attached directly to the notification email.

6. Data Model

Three Sheet tabs make up the entire database. Columns are positional — the code reads/writes them by index, so column order must not be changed without updating the corresponding read/write code.

Drivers

Col	Field	Notes
A	ID	Generated, e.g. drv_a1b2c3d4
B	Name	
C	Email	Required — used for cc'ing their invoice copy
D	PIN	4-digit, set by operations
E	Active	'Yes' / 'No' — inactive drivers don't appear in the login dropdown
F	Commission %	Per-driver, e.g. 21, 6, 25 — defaults to 20 if blank
G	Company	Legal/company name shown on their invoices
H	Next Invoice #	Auto-incrementing counter, private to this driver

Invoice Summary (one row per invoice)

Col	Field
A	Invoice # (e.g. INV-0007 — per-driver sequence)
B	Invoice ID (internal unique ID, links to Invoice Loads)
C / D / E	Driver ID / Driver Name / Company
F / G	Period Start / Period End
H	Loads (count)
I–M	Actual / Commission / Net / Tax / Driver Pay totals
N	Tax Basis ('net' or 'gross')
O	Submitted At
P	Emailed ('Yes'/'No')

Col	Field
Q	PDF Link
R	Edited At (blank until first edit)

Invoice Loads (one row per load)

Col	Field
A / B / C	Invoice ID / Driver ID / Driver Name
D	Load Date
E / F	Ref Type (TL / BOL / RM / Other) / Ref Number
G / H	Pickup / Drop-off(s) (joined with ' ' if multiple)
I	Province (destination — decides tax rate)
J	Actual Amount
K / L	Commission % / Commission Amount
M	Net After Commission
N / O	Tax Rate / Tax Amount
P	Driver Pay

7. Business Logic

7.1 Commission

Every driver has an individual commission percentage (not a fixed company-wide rate). It's applied to each load's Actual Amount:

```
commission = price * (driver.commission / 100)
net = price - commission
```

7.2 Tax

Tax is based on the load's *destination* province, not the origin — a load can start in either province, but only where it ends determines the rate:

- Ending in **Quebec** → 5%
- Ending in **Ontario** → 13%

Tax is calculated on the *net* amount (after commission) by default — this matches how an owner-operator typically bills: the commission was never their revenue, so tax applies only to what they're actually invoicing for. This is configurable per system ("net" vs "gross") in Operations → Settings.

```
taxable = (taxBasis === 'gross') ? price : net;
tax = taxable * taxRateForProvince;
driverPay = net + tax;
```

7.3 Invoice numbering

Each driver has their own independent invoice counter (stored on their row in the Drivers sheet), not a single global sequence. `nextInvoiceNumberFor_(driverId)` reads the counter, returns it, and increments it — so driver A's INV-0005 has no relationship to driver B's invoice numbers.

7.4 Editing an invoice

Editing doesn't create a new invoice — `updateInvoice( )` finds the existing summary row by Invoice ID, deletes and re-writes that invoice's load rows, regenerates the PDF, resends the email (subject line prefixed "[Updated]"), and overwrites the summary row in place — preserving the original Invoice # and Submitted At timestamp, while recording a new Edited At value.

Known tradeoff: an edit recalculates commission and tax using the driver's *current* commission rate at the time of the edit — not whatever rate was in effect when the invoice was first submitted. If a driver's rate changes and an old invoice is later edited, the edited figures will reflect the new rate.

8. Security Model

This system uses two independent PIN layers, both enforced entirely client-side:

- A single, shared **Operations PIN**, gating the whole Operations Portal.
- An individual **Driver PIN** per driver, gating that driver's own submissions and invoice history.

Important: the Apps Script Web App itself is deployed with "Who has access: Anyone" — meaning the underlying API endpoint has no authentication of its own. The PINs control what the *interface* shows, not what the *API* will accept — anyone who obtained the deployment URL directly could call it and read or write data without going through any PIN screen at all. This is a deliberate, accepted tradeoff appropriate for a small internal business tool, not a substitute for real authentication if this system's scope or sensitivity grows.

Generated PDFs are shared as "Anyone with the link — View" on Google Drive, so they can be opened without requiring the recipient to have a Google account.

9. Deployment Model

- The frontend is a single static HTML file — hosted anywhere that serves static files (Netlify, GitHub Pages, etc.). No server-side code, no build step.
- The backend is deployed as an Apps Script Web App (Execute as: Me, Access: Anyone), which produces a stable `/exec` URL. That URL is pasted into `CONFIG.SCRIPT_URL` at the top of the HTML file.
- To ship a backend code change, redeploy the *same* deployment as a "New version" (Deploy → Manage deployments → pencil icon) rather than creating a brand-new deployment — this preserves the existing URL, so the frontend never needs to change.

10. Known Limitations

- No real concurrency control — if two people write to the same row at the exact same moment, the last write wins. Acceptable at this system's current scale (a small driver roster, biweekly submissions), but not something that would hold up at much larger scale.
- PIN-based access is convenience-level protection, not cryptographic security (see Section 8).
- Apps Script cold starts can add a few seconds of latency on the first request of a session, more noticeable on slower mobile connections.
- Editing an invoice uses the driver's current commission rate, not the historical rate at original submission (see Section 7.4).